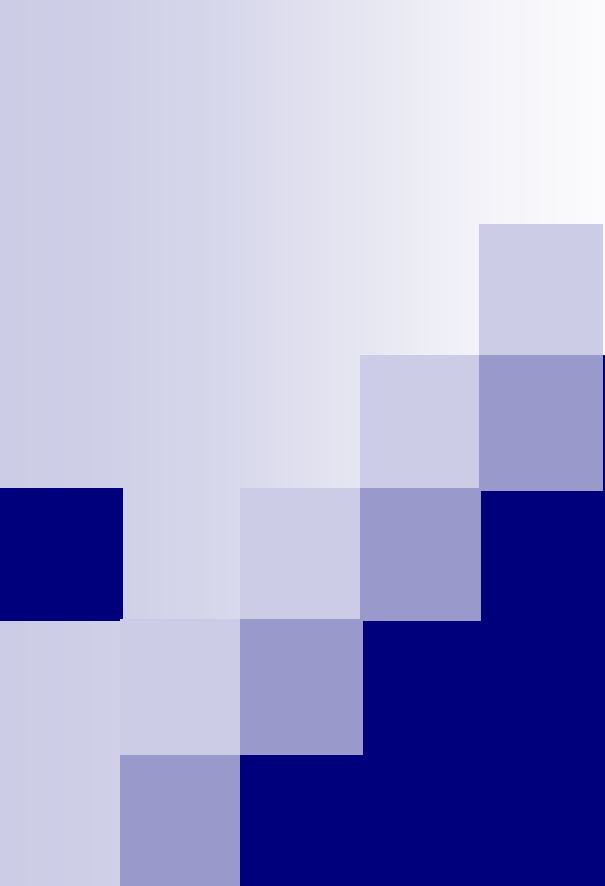




RISC and CISC

Lecture 7



Overview

- History of CISC and RISC
- CISC and RISC
 - Philosophy
 - Attributes and disadvantages
- Summation

History of RISC/CISC

- 1950s IBM instituted a research program
- 1964 Release of System/360
- Mid-1970s improved measurement tools demonstrated on CISC
- 1975 801 project initiated at IBM's Watson Research Center
- 1979 32-bit RISC microprocessor (801) developed led by Joel Birnbaum
- 1984 MIPS developed at Stanford, as well as projects done at Berkeley
- 1988 RISC processors had taken over high-end of the workstation market
- Early 1990s IBM's POWER (***P**erformance **O**ptimization **W**ith **E**nhanced **R**ISC*) architecture introduced w/ the RISC System/6k
 - AIM (Apple, IBM, Motorola) alliance formed, resulting in PowerPC



What is CISC?

- CISC is an acronym for Complex Instruction Set Computer and are chips that are easy to program and which make efficient use of memory. Since the earliest machines were programmed in assembly language and memory was slow and expensive, the CISC philosophy made sense, and was commonly implemented in such large computers as the PDP-11 and the DECsystem 10 and 20 machines.
- Most common microprocessor designs such as the Intel 80x86 and Motorola 68K series followed the CISC philosophy.
- But recent changes in software and hardware technology have forced a re-examination of CISC and many modern CISC processors are hybrids, implementing many RISC principles.
- CISC was developed to make compiler development simpler. It shifts most of the burden of generating machine instructions to the processor. For example, instead of having to make a compiler write long machine instructions to calculate a square-root, a CISC processor would have a built-in ability to do this.

CISC Attributes

The design constraints that led to the development of CISC (small amounts of slow memory and fact that most early machines were programmed in assembly language) give CISC instructions sets some common characteristics:

- A 2-operand format, where instructions have a source and a destination. Register to register, register to memory, and memory to register commands. Multiple addressing modes for memory, including specialized modes for indexing through arrays
- Variable length instructions where the length often varies according to the addressing mode
- Instructions which require multiple clock cycles to execute.

E.g. Pentium is considered a modern CISC processor



Most CISC hardware architectures have several characteristics in common:

- Complex instruction-decoding logic, driven by the need for a single instruction to support multiple addressing modes.
- A small number of general purpose registers. This is the direct result of having instructions which can operate directly on memory and the limited amount of chip space not dedicated to instruction decoding, execution, and microcode storage.
- Several special purpose registers. Many CISC designs set aside special registers for the stack pointer, interrupt handling, and so on. This can simplify the hardware design somewhat, at the expense of making the instruction set more complex.
- A 'Condition code' register which is set as a side-effect of most instructions. This register reflects whether the result of the last operation is less than, equal to, or greater than zero and records if certain error conditions occur.



At the time of their initial development, CISC machines used available technologies to optimize computer performance.

- Microprogramniing is as easy as assembly language to implement, and much less expensive than hardwiring a control unit.
- The ease of microcoding new instructions allowed designers to make CISC machines upwardly compatible: a new computer could run the same programs as earlier computers because the new computer would contain a superset of the instructions of the earlier computers.
- As each instruction became more capable, fewer instructions could be used to implement a given task. This made more efficient use of the relatively slow main memory.
- Because microprogram instruction sets can be written to match the constructs of high-level languages, the compiler does not have to be as complicated.

CISC Disadvantages

Designers soon realised that the CISC philosophy had its own problems, including:

- Earlier generations of a processor family generally were contained as a subset in every new version - so instruction set & chip hardware become more complex with each generation of computers.
- So that as many instructions as possible could be stored in memory with the least possible wasted space, individual instructions could be of almost any length - this means that different instructions will take different amounts of clock time to execute, slowing down the overall performance of the machine.
- Many specialized instructions aren't used frequently enough to justify their existence -approximately 20% of the available instructions are used in a typical program.
- CISC instructions typically set the condition codes as a side effect of the instruction. Not only does setting the condition codes take time, but programmers have to remember to examine the condition code bits before a subsequent instruction changes them.

What is RISC?

- **RISC?**

RISC, or *Reduced Instruction Set Computer*, is a type of microprocessor architecture that utilizes a small, highly-optimized set of instructions, rather than a more specialized set of instructions often found in other types of architectures.

- **History**

The first RISC projects came from IBM, Stanford, and UC-Berkeley in the late 70s and early 80s. The IBM 801, Stanford MIPS, and Berkeley RISC 1 and 2 were all designed with a similar philosophy which has become known as RISC. Certain design features have been characteristic of most RISC processors:

- *one cycle execution time*: RISC processors have a CPI (clock per instruction) of one cycle. This is due to the optimization of each instruction on the CPU and a technique called PIPELINING
- *pipelining*: a technique that allows for simultaneous execution of parts, or stages, of instructions to more efficiently process instructions;
- *large number of registers*: the RISC design philosophy generally incorporates a larger number of registers to prevent in large amounts of interactions with memory

RISC Attributes

The main characteristics of CISC microprocessors are:

- Extensive instructions.
- Complex and efficient machine instructions.
- Microencoding of the machine instructions.
- Extensive addressing capabilities for memory operations.
- Relatively few registers.

In comparison, RISC processors are more or less the opposite of the above:

- Reduced instruction set.
- Less complex, simple instructions.
- Hardwired control unit and machine instructions.
- Few addressing schemes for memory operands with only two basic instructions, LOAD and
- STORE
- Many symmetric registers which are organised into a register file.

RISC Pipelining

- Most instructions are register to register
- Two phases of execution
 - I: Instruction fetch
 - E: Execute
 - ALU operation with register input and output
- For load and store
 - I: Instruction fetch
 - E: Execute
 - Calculate memory address
 - D: Memory
 - Register to memory or memory to register operation

Effects of Pipelining

Load $rA \leftarrow M$
 Load $rB \leftarrow M$
 Add $rC \leftarrow rA + rB$
 Store $M \leftarrow rC$
 Branch X

I	E	D								
			I	E	D					
						I	E			
								I	E	D
									I	E

(a) Sequential execution

Load $rA \leftarrow M$
 Load $rB \leftarrow M$
 Add $rC \leftarrow rA + rB$
 Store $M \leftarrow rC$
 Branch X
 NOOP

I	E	D								
	I		E	D						
			I		E					
					I	E	D			
						I		E		
								I	E	

(b) Two-stage pipelined timing

Load $rA \leftarrow M$
 Load $rB \leftarrow M$
 NOOP
 Add $rC \leftarrow rA + rB$
 Store $M \leftarrow rC$
 Branch X
 NOOP

I	E	D								
	I	E	D							
		I	E							
			I	E						
				I	E	D				
					I	E				
						I	E			

(c) Three-stage pipelined timing

Load $rA \leftarrow M$
 Load $rB \leftarrow M$
 NOOP
 NOOP
 Add $rC \leftarrow rA + rB$
 Store $M \leftarrow rC$
 Branch X
 NOOP
 NOOP

I	E ₁	E ₂	D							
	I	E ₁	E ₂	D						
		I	E ₁	E ₂						
			I	E ₁	E ₂					
				I	E ₁	E ₂	D			
					I	E ₁	E ₂			
						I	E ₁	E ₂		
							I	E ₁	E ₂	

(d) Four-stage pipelined timing



RISC Disadvantages

- There is still considerable controversy among experts about the ultimate value of RISC architectures. Its proponents argue that RISC machines are both cheaper and faster, and are therefore the machines of the future.
- However, by making the hardware simpler, RISC architectures put a greater burden on the software. Is this worth the trouble because conventional microprocessors are becoming increasingly fast and cheap anyway?



CISC versus RISC

CISC

- Emphasis on hardware
- Includes multi-clock
complex instructions
- Memory-to-memory:
"LOAD" and "STORE"
incorporated in instructions
- Small code sizes,
high cycles per second
- Transistors used for storing
complex instructions

RISC

- Emphasis on software
- Single-clock,
reduced instruction only
- Register to register:
"LOAD" and "STORE"
are independent instructions
- Low cycles per second,
large code sizes
- Spends more transistors
on memory registers



CISC vs RISC code

CISC Approach

- MULT 2:3, 5:2

RISC Approach

- LOAD A, 2:3
LOAD B, 5:2
PROD A, B
STORE 2:3, A



Performance

- The CISC approach attempts to minimize the number of instructions per program, sacrificing the number of cycles per instruction.
- RISC does the opposite, reducing the cycles per instruction at the cost of the number of instructions per program.



Summation

- As memory speed increased, and high-level languages displaced assembly language, the major reasons for CISC began to disappear, and computer designers began to look at ways computer performance could be optimized beyond just making faster hardware.
- One of their key realizations was that a sequence of simple instructions produces the same results as a sequence of complex instructions, but can be implemented with a simpler (and faster) hardware design. (Assuming that memory can keep up.) RISC (Reduced Instruction Set Computers) processors were the result.
- CISC and RISC implementations are becoming more and more alike. Many of today's RISC chips support as many instructions as yesterday's CISC chips. And today's CISC chips use many techniques formerly associated with RISC chips.
- To some extent, the argument is becoming moot because CISC and RISC implementations are becoming more and more alike. Many of today's RISC chips support as many instructions as yesterday's CISC chips. And today's CISC chips use many techniques formerly associated with RISC chips.



Modern Day Advancement

- **CISC and RISC Convergence**

State of the art processor technology has changed significantly since RISC chips were first introduced in the early '80s. Because a number of advancements are used by both RISC *and* CISC processors, the lines between the two architectures have begun to blur. In fact, the two architectures almost seem to have adopted the strategies of the other. Because processor speeds have increased, CISC chips are now able to execute more than one instruction within a single clock. This also allows CISC chips to make use of pipelining. With other technological improvements, it is now possible to fit many more transistors on a single chip.

- 
- This gives RISC processors enough space to incorporate more complicated, CISC-like commands. RISC chips also make use of more complicated hardware, making use of extra function units for superscalar execution. All of these factors have led some groups to argue that we are now in a "post-RISC" era, in which the two styles have become so similar that distinguishing between them is no longer relevant. However, it should be noted that RISC chips still retain some important traits. RISC chips strictly utilize uniform, single-cycle instructions. They also retain the register-to-register, load/store architecture. And despite their extended instruction sets, RISC chips still have a large number of general purpose registers.